

Week 9 Lab: PostgreSQL & SQLAlchemy

Course: Web Design and Programming Instructor: M.Sc. Le Duy Khanh Audience: Data Science and AI students

Goal of this lab: In Week 7 your CRUD API stored data in a Python list, so everything was lost on restart. Today you move it onto a real **PostgreSQL** database using **SQLModel**. By the end of the session you will have a **Hero API** with teams, heroes and missions that **survives a server restart**, plus a seed script and a first **Alembic migration**.

Overview

Part	Topic	Time	Slides covered
0	Install and start PostgreSQL locally	15'	Running PostgreSQL Locally
1	SQL warm-up in <code>psql</code> : tables, PK, FK, JOIN	20'	Relational Databases
2	Project setup and <code>DATABASE_URL</code>	10'	Connection String, <code>DATABASE_URL</code>
3	Define models: tables + API schemas	20'	ORM & SQLAlchemy, Defining Models
4	Engine, <code>create_all</code> , <code>Session</code> per request	15'	Engine, Session & Tables
5	CRUD endpoints for heroes	30'	CRUD with SQLAlchemy, Worked Example
6	Teams, filtering/pagination, 404 and 409	20'	SQL vs SQLAlchemy vs pandas, Errors
7	Many-to-many: missions	15'	Many-to-Many: Link Table
8	Seed script	10'	Worked Example: Seed Script
9	Migrations with Alembic	20'	Migrations with Alembic
10	Project integration (group work)	rest / homework	Lab 3
—	Review questions and self-check	5'	Review & Summary

Rules for today

- Do the parts **in order**: each one builds on the previous.
- Every part ends with a  **Checkpoint**. Do not move on until the checkpoint passes.
- Reference solutions are hidden in collapsible **"Solution"** blocks. Try first, open them only when stuck.
- Keep `echo=True` on the engine all session and **read the SQL it prints**: that is the point of the lab.

What to submit (end of the session):

1. Your `hero-api/` folder (zip or Git link) containing `app/`, `migrations/`, `alembic.ini`.
 2. A file `answers.md` with your answers to every  **Question** in this lab.
 3. Screenshots of: the `\dt` output (Part 4), a successful `GET /heroes` after a restart (Part 5), and `alembic history` (Part 9).
-

Part 0 — Install and Start PostgreSQL Locally (15')

PostgreSQL runs as a **server** (a background service listening on port `5432`). You talk to it with a **client**: `psql` on the command line, or your Python app.

0.1 Install and start the server

macOS (Homebrew)

```
brew install postgresql@16
brew services start postgresql@16

# postgresql@16 is "keg-only": add its tools (psql, createdb...) to your PATH
echo 'export PATH="$(brew --prefix postgresql@16)/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Ubuntu / Debian

```
sudo apt update
sudo apt install postgresql
sudo service postgresql start
```

Windows

1. Download the installer from <https://www.postgresql.org/download/windows/> (version 16).
2. Keep the default components (Server, pgAdmin, Command Line Tools). Choose a password for the `postgres` superuser and **write it down**. Keep port `5432`.
3. Add `C:\Program Files\PostgreSQL\16\bin` to your `PATH` so that `psql` works in a new terminal.

Check the installation:

```
psql --version          # psql (PostgreSQL) 16.x
```

0.2 Create the application user and database

Open `psql` as the administrator:

OS	Command
macOS (Homebrew)	<code>psql postgres</code>
Ubuntu / Debian	<code>sudo -u postgres psql</code>
Windows	<code>psql -U postgres</code> (enter the password from the installer)

Then, at the `postgres=#` prompt:

```
CREATE USER app WITH PASSWORD 'secret';
CREATE DATABASE appdb OWNER app;
\q
```

Connect again, this time **as the app user, using a connection string**:

```
psql "postgresql://app:secret@localhost:5432/appdb"
```

✅ **Checkpoint 0:** you see the prompt `appdb=>`. Type `\conninfo` — it says you are connected to database `appdb` as user `app` on port `5432`.

► Troubleshooting

Part 1 — SQL Warm-up: Tables, Keys, Relationships (20')

Before the ORM writes SQL for you, write some yourself. Stay in `psql` connected as `app` to `appdb`.

1.1 Create two related tables

```
CREATE TABLE team (  
  id          SERIAL PRIMARY KEY,  
  name        VARCHAR NOT NULL UNIQUE,  
  headquarters VARCHAR NOT NULL  
);  
  
CREATE TABLE hero (  
  id          SERIAL PRIMARY KEY,  
  name        VARCHAR NOT NULL,  
  age         INTEGER,  
  team_id     INTEGER REFERENCES team(id)  
);
```

Inspect them:

```
\dt          -- list tables  
\d hero      -- describe one table: columns, types, keys, indexes
```

1.2 Insert and query

```
INSERT INTO team (name, headquarters)  
VALUES ('Avengers', 'New York'), ('X-Men', 'Westchester');  
  
INSERT INTO hero (name, age, team_id)  
VALUES ('Tony', 45, 1), ('Natasha', 35, 1), ('Logan', 150, 2), ('Peter', 16, NULL);  
  
SELECT * FROM hero;
```

Task 1. Write SQL queries for:

1. All heroes aged 18 or older, oldest first.
2. Each hero's name next to their team's name (heroes without a team must also appear).
3. The number of heroes per team.
4. The 2nd page of heroes when a page has 2 rows, ordered by `id`.

► Solution

1.3 Let the database protect your data

Run each statement and read the error message:

```
INSERT INTO team (name, headquarters) VALUES ('Avengers', 'Los Angeles');  
INSERT INTO hero (name, team_id) VALUES ('Ghost', 99);  
INSERT INTO hero (age) VALUES (30);  
DELETE FROM team WHERE id = 1;
```

? **Question 1.** For each of the four statements, which constraint blocked it (PRIMARY KEY , UNIQUE , NOT NULL , FOREIGN KEY) and why?

? **Question 2.** The relationship team → hero is **one-to-many**. Why is the foreign key on hero and not on team ?

? **Question 3.** Heroes can go on many missions and a mission has many heroes (**many-to-many**). Sketch the tables you need (names, columns, PK, FK). Hint: you need a **link table**.

1.4 Clean up

SQLModel will create these tables itself later, so drop them now:

```
DROP TABLE hero;
DROP TABLE team;
\q
```

✓ **Checkpoint 1:** \dt shows *Did not find any relations.*

Part 2 — Project Setup and DATABASE_URL (10')

2.1 Create the project

```
mkdir hero-api && cd hero-api
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install "fastapi[standard]" sqlalchemy "psycopg[binary]" alembic
```

Create this structure (empty files for now):

```
hero-api/
├── app/
│   ├── __init__.py
│   ├── database.py      # engine, get_session, SessionDep
│   ├── models.py        # tables and API schemas
│   ├── main.py          # FastAPI app and endpoints
│   └── seed.py          # sample data
└── .gitignore           # .venv/ __pycache__/ *.db
```

2.2 The connection string

A database URL has the shape:

```
dialect+driver://user:password@host:port/dbname
```

Database	URL
PostgreSQL (psycopg 3 driver)	<code>postgresql+psycopg://app:secret@localhost:5432/appdb</code>
SQLite (a file, no server)	<code>sqlite:///./app.db</code>

⚠ In Python, write `postgresql+psycopg://`. Plain `postgresql://` makes SQLAlchemy look for the old `psycopg2` driver, which we did not install (`ModuleNotFoundError: No module named 'psycopg2'`). The `psql` client, on the other hand, only understands plain `postgresql://`.

2.3 Never hardcode credentials

Set the URL as an **environment variable** in the terminal where you will run the app:

```
# macOS / Linux
export DATABASE_URL="postgresql+psycopg://app:secret@localhost:5432/appdb"

# Windows PowerShell
$env:DATABASE_URL = "postgresql+psycopg://app:secret@localhost:5432/appdb"
```

The variable only lives in **that** terminal. If you open a new one, set it again (or add the line to your shell profile).

❓ **Question 4.** Why read the URL from an environment variable instead of writing it in `database.py`? Give two reasons.

✅ **Checkpoint 2:** `echo $DATABASE_URL` (PowerShell: `echo $env:DATABASE_URL`) prints the URL, and `python -c "import sqlalchemy, psycopg"` runs without error.

Part 3 — Define the Models (20')

`SQLModel` = **SQLAlchemy** (tables, engine, sessions, queries) + **Pydantic** (types, validation, JSON). One class can be **both** a database table and an API schema.

3.1 Tables

Task 3.1. In `app/models.py`, define two **tables** matching your warm-up SQL from Part 1:

- `Team`: `id` (primary key, generated by the DB), `name` (indexed, **unique**), `headquarters`.
- `Hero`: `id` (primary key), `name` (indexed), `age` (optional), `secret_name` (required), `team_id` (optional foreign key to `team.id`).

- A **bidirectional relationship**: `team.heroes` (list of heroes) and `hero.team` (the team or `None`).

Starter:

```
from sqlmodel import Field, Relationship, SQLModel

class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    # TODO: name, headquarters
    # TODO: heroes relationship

class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    # TODO: name, age, secret_name, team_id
    # TODO: team relationship
```

► Solution

? **Question 5.** Why is `id` typed `int | None` with `default=None`, when every row in the database has an id?

? **Question 6.** Which attributes of `Hero` become **columns**, and which one does not? What is `back_populates` for?

3.2 One model, two roles: API schemas

The API must **not** return `secret_name`, the client must **not** send `id`, and a `PATCH` body has **all fields optional**. Split the models into a shared base + a table + schemas.

Task 3.2. Refactor `models.py` to the pattern below, for both heroes and teams:

Class	<code>table=True</code> ?	Role	Fields
<code>HeroBase</code>	no	shared fields	<code>name</code> , <code>age</code> , <code>team_id</code>
<code>Hero</code>	yes	the table	base + <code>id</code> , <code>secret_name</code> , <code>team</code>
<code>HeroCreate</code>	no	<code>POST</code> body	base + <code>secret_name</code>
<code>HeroPublic</code>	no	<code>response_mode</code> 1	base + <code>id</code> (required <code>int</code>)
<code>HeroUpdate</code>	no	<code>PATCH</code> body	<code>name</code> , <code>age</code> , <code>team_id</code> , <code>secret_name</code> , all optional

Class	table=True ?	Role	Fields
TeamBase	no	shared fields	name , headquarters
Team	yes	the table	base + id , heroes
TeamCreate	no	POST body	base
TeamPublic	no	response_mode 1	base + id

► Solution

✓ **Checkpoint 3:** `python -c "from app.models import Hero, HeroPublic; print(Hero.__tablename__)"` prints `hero`.

Part 4 — Engine, Tables and Session (15')

4.1 The engine and the session dependency

Task 4.1. In `app/database.py`:

1. Read `DATABASE_URL` from the environment, falling back to `sqlite:///./app.db`.
2. Create **one** engine with `echo=True`. For SQLite only, pass `connect_args={"check_same_thread": False}`.
3. Write the `get_session()` **yield-dependency** (Week 8) and the `SessionDep` type alias.

► Solution

4.2 Create the tables at startup

Task 4.2. In `app/main.py`, create the app with a **lifespan** function that calls `SQLModel.metadata.create_all(engine)`.

► Solution

Run it:

```
fastapi dev app/main.py
```

Read the terminal: thanks to `echo=True` you will see `CREATE TABLE team (...)` and `CREATE TABLE hero (...)`. In a **second terminal**, look at the database:

```
psql "postgresql://app:secret@localhost:5432/appdb" -c "\dt"
psql "postgresql://app:secret@localhost:5432/appdb" -c "\d hero"
```

? **Question 7.** Compare the `CREATE TABLE hero` printed by SQLAlchemy with the one you wrote by hand in Part 1. List the differences (types, `NOT NULL`, indexes, constraints).

? **Question 8.** Stop and restart the server. Is `CREATE TABLE` printed again? Why? What does `create_all` do when a table already exists?

? **Question 9.** `create_all` only knows about models that have been **imported**. Which line in `main.py` makes sure `Hero` and `Team` are registered?

✓ **Checkpoint 4:** `\dt` lists `hero` and `team`. 📸 Screenshot it.

Part 5 — CRUD Endpoints for Heroes (30')

Op	SQLModel	SQL
Create	<code>session.add(obj) → session.commit() → session.refresh(obj)</code>	<code>INSERT</code>
Read one	<code>session.get(Hero, id) → object or None</code>	<code>SELECT ... WHERE id = ?</code>
Read many	<code>session.exec(select(Hero).where(...)).all()</code>	<code>SELECT ... WHERE ...</code>
Update	<code>load → set attributes → add → commit</code>	<code>UPDATE</code>
Delete	<code>session.delete(obj) → commit</code>	<code>DELETE</code>

Task 5. In `app/main.py`, implement:

Method & path	Body	Response	Status codes
<code>POST /heroes</code>	<code>HeroCreate</code>	<code>HeroPublic</code>	201
<code>GET /heroes?offset=0&limit=10</code>	—	<code>list[HeroPublic]</code>	200 ; <code>limit ≤ 100</code>
<code>GET /heroes/{hero_id}</code>	—	<code>HeroPublic</code>	200 , 404 if missing
<code>PATCH /heroes/{hero_id}</code>	<code>HeroUpdate</code>	<code>HeroPublic</code>	200 , 404 if missing
<code>DELETE /heroes/{hero_id}</code>	—	—	204 , 404 if missing

Hints:

- `Hero.model_validate(hero_in)` turns a `HeroCreate` into a `Hero` table object.
- For `PATCH`, only change the fields the client actually sent:
`hero_in.model_dump(exclude_unset=True)`, then `hero.sqlmodel_update(...)`.
- Always set `response_model=HeroPublic` so `secret_name` never leaves the server.

Starter:

```
@app.post("/heroes", response_model=HeroPublic, status_code=201)
def create_hero(hero_in: HeroCreate, session: SessionDep):
    # TODO: convert, add, commit, refresh, return
    ...

@app.get("/heroes", response_model=list[HeroPublic])
def list_heroes(session: SessionDep, offset: int = 0,
                limit: int = Query(default=10, le=100)):
    # TODO: select, order by id, offset, limit
    ...
```

► Solution

5.1 Test it

Open <http://127.0.0.1:8000/docs> and try every endpoint, or use `curl`:

```
curl -X POST localhost:8000/heroes -H "Content-Type: application/json" \
     -d '{"name": "Peter", "age": 16, "secret_name": "Spider-Man"}'
curl "localhost:8000/heroes?limit=5"
curl -X PATCH localhost:8000/heroes/1 -H "Content-Type: application/json" -d '{"age": 17}'
curl -i localhost:8000/heroes/999
curl -i -X DELETE localhost:8000/heroes/1
```

Watch the server terminal while you call each endpoint.

? Question 10. Comment out `session.commit()` in `create_hero` and create a hero. What does the response look like, and is the row in the database (`SELECT * FROM hero;`)? Put the line back. What does `add()` do on its own, and why do we need `refresh()`?

? Question 11. Which SQL statement does `echo=True` print for `PATCH` with body `{"age": 17}`? Does it update every column or only `age`? Why?

? Question 12. Look at the JSON returned by `GET /heroes/{id}`. Is `secret_name` there? Which line of code is responsible?

5.2 The whole point: persistence

Create 3 heroes. **Stop the server (Ctrl+C) and start it again.** Call `GET /heroes`.

✅ **Checkpoint 5:** the heroes are still there after the restart (unlike Week 7). 📸 Screenshot it. All five endpoints return the status codes in the table above.

Part 6 — Teams, Filtering, Pagination and Errors (20')

6.1 Team endpoints with a 409 Conflict

`Team.name` is `unique`. Inserting a duplicate makes PostgreSQL raise an error, which SQLAlchemy turns into `sqlalchemy.exc.IntegrityError`. Unhandled, that is a `500 Internal Server Error` — but it is the **client's** mistake, so we should answer `409 Conflict`.

Task 6.1. Implement:

- `POST /teams` (`TeamCreate` → `TeamPublic`, `201`). On `IntegrityError`: `session.rollback()` and raise `HTTPException(409)`.
- `GET /teams` → `list[TeamPublic]`.
- `GET /teams/{team_id}/heroes` → `list[HeroPublic]`, `404` if the team does not exist. Use the relationship `team.heroes` — no `select` needed.

► Solution

Test: create the team `Avengers` twice → the second call must return `409`, not `500`.

6.2 Filter and sort in the database

SQL	SQLModel	pandas
<code>WHERE age >= 18</code>	<code>.where(Hero.age >= 18)</code>	<code>df[df.age >= 18]</code>
<code>ORDER BY age DESC</code>	<code>.order_by(Hero.age.desc())</code>	<code>df.sort_values("age", ascending=False)</code>
<code>LIMIT 10 OFFSET 20</code>	<code>.offset(20).limit(10)</code>	<code>df.iloc[20:30]</code>

Task 6.2. Extend `GET /heroes` with optional query parameters:

- `min_age: int | None` — only heroes with `age >= min_age`
- `team_id: int | None` — only heroes of that team
- `name: str | None` — heroes whose name contains this text, case-insensitive (hint: `Hero.name.ilike(f"%{name}%")`)

Build the statement step by step: start from `select(Hero)` and add a `.where(...)` only when the parameter is not `None`.

► Solution

? **Question 13.** Call `GET /heroes?min_age=18&team_id=1` and copy the `SELECT` printed by `echo=True`. Where do the values `18` and `1` appear? Why is this safe against **SQL injection**?

? **Question 14.** Why filter in the database instead of `[h for h in session.exec(select(Hero)).all() if h.age >= 18]`?

6.3 Challenge: invalid foreign keys

Try `POST /heroes` with `"team_id": 999`, and `PATCH /heroes/1` with `{"team_id": 999}`. What happens? Make both endpoints return `404 Team not found` instead (check with `session.get(Team, ...)` before committing).

✓ **Checkpoint 6:** duplicate team → `409`; unknown team id → `404`; the filters combine correctly.

Part 7 — Many-to-Many: Missions (15')

A hero can go on many missions and a mission has many heroes. As in Question 3, this needs a **link table** with a **composite primary key** made of two foreign keys.

Task 7.

- In `models.py`, add `HeroMissionLink` (table) and `Mission` (table: `id`, `title`), plus `MissionCreate` and `MissionPublic`.
 - Define `HeroMissionLink` **above** `Hero`, because `Hero` refers to it.
 - Add `missions: list["Mission"] = Relationship(back_populates="heroes", link_model=HeroMissionLink)` to `Hero`, and the mirror `heroes` relationship to `Mission`.
- In `main.py`, add:
 - `POST /missions` → `MissionPublic` (`201`)
 - `POST /heroes/{hero_id}/missions/{mission_id}` → assign a hero to a mission (`204`, `404` if either is missing; do nothing if already assigned)
 - `GET /heroes/{hero_id}/missions` → `list[MissionPublic]`
- Restart the server.

► Solution — `models.py` (additions)

► Solution — `main.py` (additions)

Check the link table after assigning a mission:

```
psql "postgresql://app:secret@localhost:5432/appdb" -c "\d heromissionlink"
psql "postgresql://app:secret@localhost:5432/appdb" -c "SELECT * FROM heromissionlink;"
```

? Question 15. On restart, `create_all` **did** create `mission` and `heromissionlink`. In Part 4 it did nothing for `hero`. What is the rule?

✓ Checkpoint 7: after assigning hero 1 to mission 1 twice, `heromissionlink` contains exactly **one** row `(1, 1)`.

Part 8 — Seed Script (10')

Your frontend and dashboard (upcoming weeks) need realistic data. A **seed script** fills the database in one command — the database version of loading a DataFrame from a CSV.

Task 8. Write `app/seed.py`:

- Create the tables (`create_all`).
- If there is already at least one team, print a message and stop (so the script can be run twice safely).
- Otherwise insert 2 teams, at least 5 heroes and 2 missions, and assign some heroes to missions — **using relationships** (`team=avengers` , `missions=[...]`), not by typing ids.
- Make it runnable with `python -m app.seed`.

► Solution

To start from an empty database, drop everything first (as `app`):

```
psql "postgresql://app:secret@localhost:5432/appdb" \  
-c "DROP TABLE IF EXISTS heromissionlink, hero, mission, team CASCADE;"  
python -m app.seed  
python -m app.seed          # second run: "already seeded"  
fastapi dev app/main.py    # try every endpoint at /docs
```

? Question 16. You never set `team_id` in the seed script. Read the `echo=True` output: in which order were the `INSERT` s executed, and how did `hero.team_id` get its value?

✓ Checkpoint 8: `GET /teams/1/heroes` returns the Avengers, `GET /heroes/1/missions` returns *Battle of Sokovia*.

Part 9 — Migrations with Alembic (20')

9.1 See the problem

Task 9.1. Add a new field to `HeroBase`:

```
power: str | None = None
```

Restart the server, then:

```
psql "postgresql://app:secret@localhost:5432/appdb" -c "\d hero"
```

? Question 17. Is there a `power` column? Now call `GET /heroes`. What happens and why? Why is "drop all tables and run `create_all` again" not an acceptable fix in production?

9.2 Set up Alembic

Stop the server. From the `hero-api/` folder:

```
alembic init migrations
```

This creates `alembic.ini` and the folder `migrations/` (with `env.py`, `script.py.mako`, `versions/`).

Edit 1 — `migrations/env.py`. Tell Alembic where the database is and what the models look like. Near the top, after `config = context.config`, add:

```
import os

from sqlmodel import SQLModel

from app import models # noqa: F401 (import so the tables are registered)

config.set_main_option("sqlalchemy.url", os.environ["DATABASE_URL"])
```

and further down replace `target_metadata = None` with:

```
target_metadata = SQLModel.metadata
```

Edit 2 — `migrations/script.py.mako`. Autogenerated scripts use `SQLModel`'s own column types, so they must import `sqlmodel`. Add this line next to `import sqlalchemy as sa`:

```
import sqlmodel
```

9.3 Generate, review, apply

```
alembic revision --autogenerate -m "add hero.power"
```

Open the new file in `migrations/versions/`. **Always review it** — autogenerate compares your models with the real database and guesses.

? **Question 18.** Copy the bodies of `upgrade()` and `downgrade()`. What does each one do?

Apply it, check it, roll it back, re-apply:

```
alembic upgrade head          # apply pending migrations
psql "postgresql://app:secret@localhost:5432/appdb" -c "\d hero"      # power is there
alembic downgrade -1          # undo the last migration
psql "postgresql://app:secret@localhost:5432/appdb" -c "\d hero"      # power is gone
alembic upgrade head
alembic history               # list the revisions
alembic current               # which revision the database is at
```

Add `power: str | None = None` to `HeroUpdate` too (it is already in `HeroBase`, so `HeroPublic` returns it). Restart the server, `PATCH` a hero with `{"power": "flight"}`, and check that `GET /heroes` works again.

? **Question 19.** Where does Alembic store "which revision this database is at"? (Hint: `\dt .`) Why should the `migrations/` folder be committed to Git?

? **Question 20.** Rename `secret_name` to `alias` in the model and run `revision --autogenerate` (do **not** apply it). What did Alembic generate? Why is that dangerous for existing data, and how would you fix the script? Afterwards, delete that revision file and undo the rename.

✓ **Checkpoint 9:** `alembic history` shows your revision, `alembic current` shows `(head)`. 📸
Screenshot it.

Part 10 — Project Integration (group work, finish as homework)

Apply everything to your **group project**:

1. **Schema on paper.** Draw your tables: columns with types, primary keys, foreign keys, and every relationship labelled one-to-many or many-to-many (with its link table).
2. **Models.** Write the SQLAlchemy models: for each entity a `table=True` class plus `Create / Public / Update` schemas. Decide which fields must never be returned (passwords, internal notes...).
3. **CRUD.** Move your Week 7 in-memory endpoints onto the database with `SessionDep`, with `404` and `409` handling.
4. **Seed.** Write `seed.py` with realistic sample data (at least ~20 rows in the main table) for the frontend and dashboard.
5. **Migrations.** Initialise Alembic in the project and create a first revision.

✓ **Checkpoint 10:** a teammate can clone the repo, set `DATABASE_URL`, run `alembic upgrade head` and `python -m app.seed`, and see data at `/docs`.

Review Questions

Answer without looking, then check.

1. What is a primary key for?
2. What is the main benefit of an ORM?
3. What does `table=True` change about a `SQLModel` class?
4. After `session.add(...)`, what actually saves the data to the database?
5. What does `session.exec(select(Model)).all()` return?
6. What is Alembic for?
7. You add a field to a model and restart the app. Why is there no new column?
8. In a one-to-many Team–Hero relationship, where does the foreign key go?

► Answers

Self-Check

- ☐ I can explain PK, FK, one-to-many and many-to-many, and write the SQL for them.
- ☐ I can build a `DATABASE_URL` and read it from the environment.
- ☐ I can explain what an ORM is and what `SQLModel` combines.
- ☐ I can define models with `table=True`, `Field(...)` and `Relationship()`, and split them into `Base` / `table` / `Create` / `Public` / `Update`.
- ☐ I can set up the engine, `create_all` and `SessionDep`.
- ☐ I can write all five CRUD endpoints with `404` and `409` handling, and filter/paginate in the database.
- ☐ I know when and why to use Alembic, and I always review autogenerated migrations.

Next week (Week 10): RESTful design, automatic docs, and API tests with `pytest` + `TestClient`. Keep your `hero-api` — we will write tests against it.